

NiftyDrum

Official Documentation

Ronna Technologies

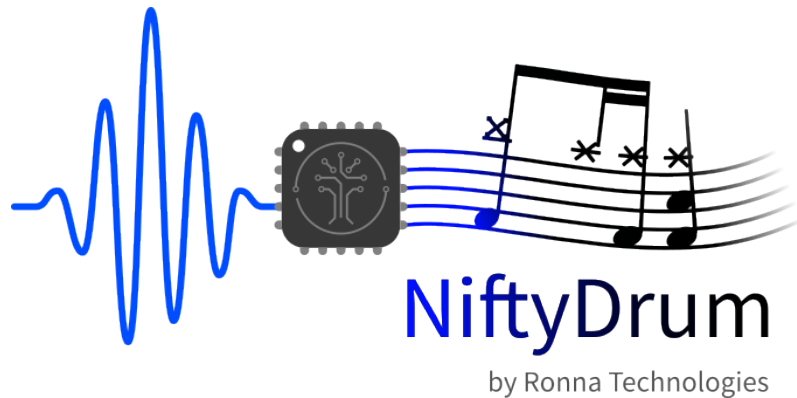
Copyright © 2026 Ronna Technologies

Table of contents

1. NiftyDrum(+)	4
1.1 Description	4
1.2 How It Works	4
1.3 What Makes the Project Special	5
1.4 Specifications	5
2. The Boards	7
2.1 NiftyDrum	7
2.2 NiftyDrum+	9
2.3 Latency	9
3. Trigger Module App	11
3.1 Configure NiftyDrum	11
3.2 How to Install the App	13
3.3 Piezo Trigger Configuration	14
3.4 Hi-Hat Pedal Configuration	15
3.5 Firmware Upgrade	16
4. Firmware Serial Protocol	19
4.1 Overview	19
4.2 Command Syntax	19
4.3 Hi-Hat Controller Parameters	22
5. Sensor SDK	24
5.1 That's a Blue Pill	24
5.2 Arduino IDE	24
5.3 Sending MIDI Notes	28
6. Sound SDK	30
6.1 WaveTooEasy	30

6.2 RonnaTech STM32F401 Core SDK	30
6.3 Firmware Updates	31
6.4 Resources	31
6.5 Example: The Default Firmware	31

1. NiftyDrum(+)



1.1 Description

- **NiftyDrum** is a trigger-to-MIDI conversion module that transforms piezo and FSR sensor inputs into MIDI messages. Connect up to 9 piezo sensors and 1 FSR (Force Sensing Resistor) to the dedicated terminal blocks, then receive MIDI data via USB-C connection.
- **NiftyDrum+** is a single, unified board combining our NiftyDrum's sensor-to-MIDI interface with a powerful sound engine. Sound samples are stored on a microSD card, and are streamed to a 3.5mm jack output.

1.2 How It Works

NiftyDrum delivers high-level MIDI performance in 4 easy steps:

- **Connect sensors:** Attach up to 9 piezo sensors and 1 FSR to the terminal blocks
- **Plug in:** Connect to your DAW, Raspberry Pi, or drum module via USB
- **Configure:** Use the web-based GUI to adjust trigger parameters, MIDI mapping, and velocity curves
- **Play:** Notes are transmitted instantly with imperceptible latency

A fifth step can be added: **Use NiftyDrum+ sound engine and get a fully-featured drum module.**

1.3 What Makes the Project Special

NiftyDrum and NiftyDrym+ both ship with their own firmware. This make NiftyDrum+ a fully-featured drum module by default. In addition to that, we provide two **Arduino**-compatible SDKs:

- [A Sensor SDK](#)
- [A Sound SDK](#)

The SDKs give you tools to develop you own drum module.

1.4 Specifications

1.4.1 Hardware

- **Piezo inputs:** 9 channels
- **FSR input:** 1 channel (hi-hat controller)
- **Connector type:** Terminal blocks
- **USB interface:** Type-C
- **Dimensions:** 65 × 56.5 mm
- **Audio:** TRS 3.5mm (1/8")

1.4.2 Performance

- **Latency:** <2.5 ms
- **Sensor Sampling rate:** >10 kHz
- **Velocity resolution:** 127 levels (full MIDI range)
- **Trigger to Sound Latency:** <10ms
- **Audio Sampling Rate:** ≥44.1kHz

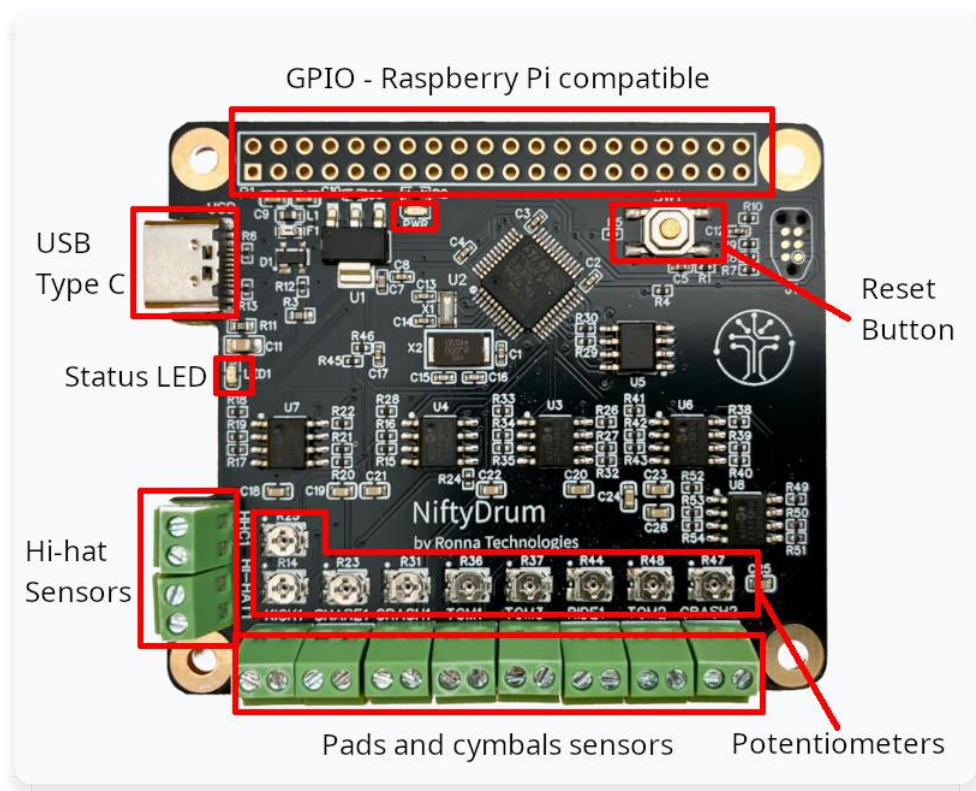
1.4.3 Software

- **Platform support:** Windows, macOS, Linux
- **User interface:** Web-based application
- **Firmware updates:** Via USB
- **MIDI output:** Note messages and Control Changes (CC)
- **Development:** Arduino SDKs

2. The Boards

2.1 NiftyDrum

The NiftyDrum board is shown in the following image.



This board features the following interfaces:

- Terminal blocks for sensor inputs
- USB Type-C port for laptop or PC connectivity
- 9 potentiometers for sensitivity adjustment
- 2 LEDs
- Reset button
- Raspberry Pi-compatible GPIO header
- 4 mounting holes

2.1.1 Connecting Sensors

The board provides 9 piezo inputs, supporting up to 9 single-zone pads, as well as 1 FSR input.

Hi-Hat Sensors

On the left side of the board, two dedicated terminal blocks are reserved for hi-hat sensors:

- **Top terminal block:** Connects to an FSR (Force Sensing Resistor) sensor for hi-hat controller input
- **Bottom terminal block:** Connects to a piezo sensor for hi-hat cymbal trigger

Standard Pads and Cymbals

The remaining eight terminal blocks, located at the bottom of the board, are for connecting regular pads and cymbals. While the board labels indicate the default firmware assignments, these inputs are fully customizable.

Important Notes

- For all terminal blocks, the ground pin is positioned on the **left-hand side**
- Nine onboard potentiometers enable hardware-level sensitivity adjustments for maximum flexibility
- If unsure about sensitivity settings, leave potentiometers at their midpoint for balanced performance

2.1.2 MIDI Outputs

The board offers two methods for transmitting MIDI notes and Control Changes:

- **USB-C port:** Outputs USB MIDI messages
- **GPIO UART pins:** Raspberry Pi GPIO-compatible interface

2.1.3 USB-C Port

Using NiftyDrum as a USB device is the recommended method for receiving MIDI messages. This configuration enables:

- Integration with DAW software for high-quality sound output from your laptop
- Control and configuration via the [official app](#)

2.1.4 Raspberry-Pi hat form factor

The board is designed with a Raspberry Pi 4 HAT form factor, ensuring seamless integration.

2.1.5 App Features

The official app provides comprehensive control over your NiftyDrum board:

- Customize MIDI note assignments for each trigger
- Design custom velocity curves per trigger
- Adjust advanced parameters including gain, threshold, scan time, mask time, and decay
- Update board firmware to the latest version

2.2 NiftyDrum+

2.3 Latency

Latency is a big topic, and the goal here is to use the least resources possible while having a competitive overall latency. Before we did into that subject, it's important to note that this parameter depends a lot on the microSD that is being used. For me, a standard SD card performed pretty well, but it really depends.

The sound board has roughly the same latency of 4.5ms, which means that, including a scan time of about 2 or 2.5ms for a trigger input, the total latency is about: $2.5 + 4.5 + 1 = 8\text{ms}$. I added 1ms to account for UART transfer between the trigger board and the sound board, as well as audio DAC latency.

In practice, I measured the latency using an oscilloscope. I wired a channel to the piezo sensor, and another one to the audio jack. I hit the pad several times and recorded the time difference between the peak voltage of the piezo and the rise of the output sound. Most of the time the latency was 8ms, as expected, but it reached 8.5 ms a few times, and was sometimes as low as 7ms. I would say that, with a very safe margin, the latency is **less than 9ms**, but I advertise the latency as being less than 10ms, just to be on the safe side, especially because it depends on the scan time.

Not that this experiment is valid only for one sound at a time. When several samples are triggered at the same time, which is never really true in practice, the samples are going to be played one after another, which will add a few hundred μs for each sample, except the first one.

3. Trigger Module App

3.1 Configure NiftyDrum

⚠ Warning

This app is designed **only for configuration purposes**. While running, it continuously transfers data between your PC and the board. For optimal performance, **close the app before playing**.

NiftyDrum is fully configurable, allowing you to adjust parameters like scan time, mask time, decay, threshold, etc. To simplify customization, a dedicated desktop application is available, compatible with Windows, Linux, and macOS.

Below are all the different commands the app can send to the board.

3.1.1 General Board Commands

Command	Description
Reset	Restart the board in bootloader mode
Serial number	Retrieve the board’s unique serial number
Version	Retrieve the current firmware version
Save current parameters	Persist current settings to the board
Load parameters	Load previously saved board parameters
Factory reset	Reset all parameters to factory defaults

3.1.2 Trigger Parameters (Per Trigger, Including Hi-Hat Cymbal)

Parameter	Description
Set/get velocity curve	Adjust or retrieve the velocity response curve
Set/get threshold	Configure the trigger activation threshold
Scan time	Set/get the trigger scan time
Mask time	Set or adjust the trigger mask time
Decay time	Adjust the decay time of the trigger
Gain	Adjust the gain level of the trigger
MIDI Note	Assign the MIDI note for the trigger

3.1.3 Hi-Hat Pedal Parameters

Parameter	Description
Update interval	Set the hi-hat pedal update frequency
Noise threshold	Ignore pedal changes below this value
Pedal offset	Determine if the hi-hat is fully closed
Velocity threshold	Set the velocity threshold for foot chick

3.1.4 How the App Works

The app simplifies customization by organizing everything logically: instruments are selected via a drop-down, while MIDI notes and velocity curves are managed separately from trigger settings for a cleaner, more efficient setup.

3.2 How to Install the App

The app is available for Windows, macOS, and Linux and can be downloaded directly from the official [NiftyDrum.com](https://niftydrum.com) website. Follow the OS-specific instructions provided on the site.

3.2.1 Windows

On Windows, the app is distributed as a `.zip` file, so no installation is required, simply extract and run it. Note that, if that's not already done, you will have to install the [Microsoft Visual C++ Redistributable package](#).

3.2.2 Linux

For Linux, the app is packaged as a `.deb` file. You can install it using your preferred package manager or by running the following command in a terminal:

Ubuntu 22.04 Ubuntu 24.04 debian

```
sudo apt install --reinstall ./NiftyDrum-1.0.0-Ubuntu-22.04.deb
```

```
sudo apt install --reinstall ./NiftyDrum-1.0.0-Ubuntu-24.04.deb
```

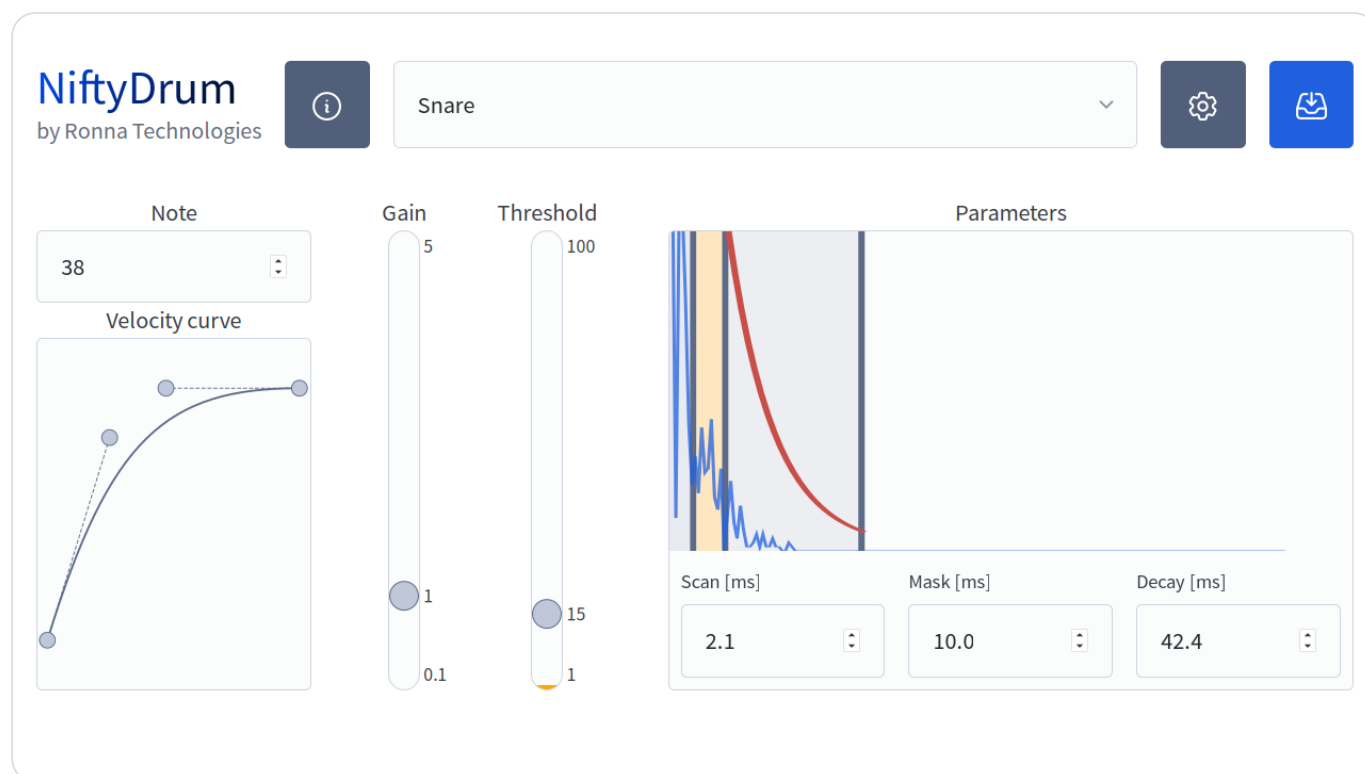
```
sudo apt install --reinstall ./NiftyDrum-1.0.0-Linux.deb
```

3.2.3 MacOS

The MacOS version of the app is provided as a `.zip` file. Just extract it and run the application. If you're using an Apple Silicon Mac, you may be prompted to install Rosetta the first time you launch the app.

3.3 Piezo Trigger Configuration

Customize each piezo trigger by selecting your desired instrument from the dropdown menu. The screenshot below illustrates the process for configuring the snare drum.



On the left side, you can assign the MIDI note for the pad, such as note 38 for the snare. Below the note input, you'll find the velocity curve editor, which uses Bézier controls for precise adjustments. You can drag and drop the endpoints and the two middle control points to shape the curve according to your needs. The horizontal axis reflects the raw MIDI velocity detected by the sensor, while the vertical axis shows the velocity value transmitted over USB. This setup lets you fine-tune the responsiveness and dynamics of your triggers.

Moving from left to right, you can adjust the trigger gain within a range of 0.1 to 5. This allows you to boost the input sensitivity of the trigger, enhancing the volume of ghost notes when the velocity curve alone isn't sufficient. It's important to note that the gain is applied after a strike has been detected, so it doesn't impact the threshold setting.

Next is the threshold setting, which sets the minimum signal level the piezo must exceed for a hit to register. This ensures only intentional strikes are detected, effectively filtering out unwanted noise and preventing false triggers.

3.4 Hi-Hat Pedal Configuration

The last item in the dropdown menu is the **hi-hat pedal configuration**. Unlike other inputs, it does not include standard parameters like scan time or mask time.

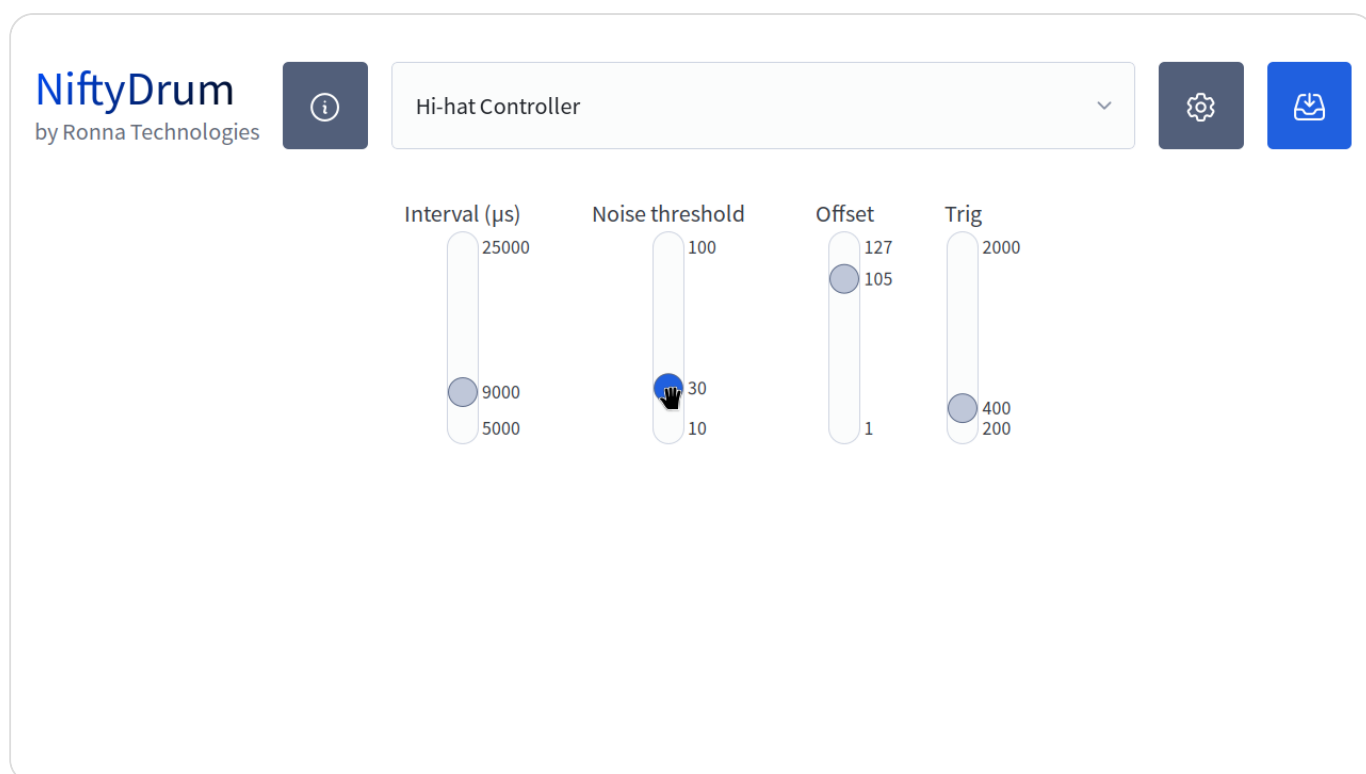
Hi-Hat Implementation Status

The hi-hat implementation is currently **experimental**. The app settings may evolve in future updates.

3.4.1 Parameters

The hi-hat pedal configuration includes four key parameters:

Parameter	Description
Update interval	Sets the frequency at which the hi-hat pedal updates.
Noise threshold	Ignores pedal changes below this value to filter out noise.
Pedal offset	Determines whether the hi-hat is fully closed.
Velocity threshold (Trig)	Sets the sensitivity threshold for triggering a foot chick note.



3.4.2 CC Message Behavior

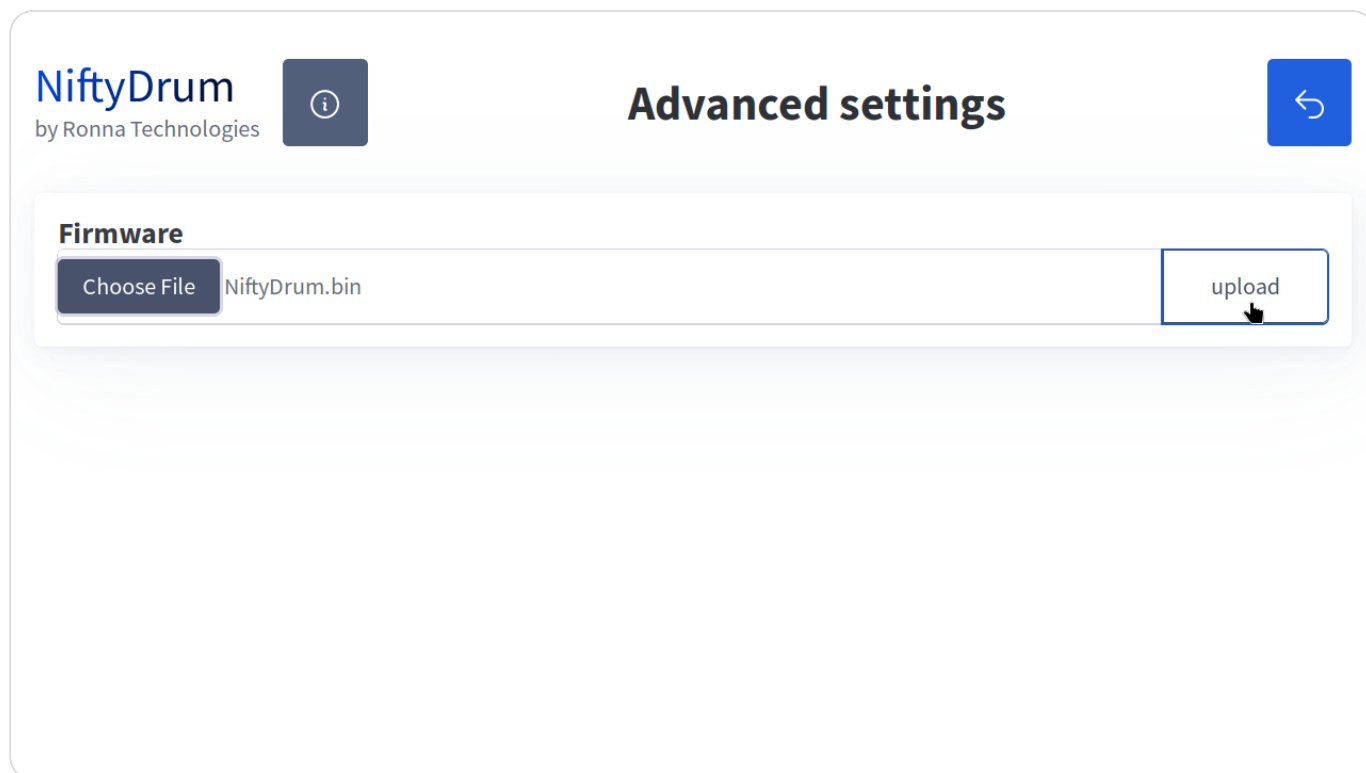
- The **Update interval** defines the minimum delay (in microseconds) between two CC messages, typically set to **10ms**.
- The **Noise threshold** sets the minimum change in value required to send a CC message.

3.4.3 Foot Chick Note Trigger

The **Pedal offset** and **Velocity threshold (Trig)** determine whether a foot chick note is sent, with the latter acting as a sensitivity threshold.

3.5 Firmware Upgrade

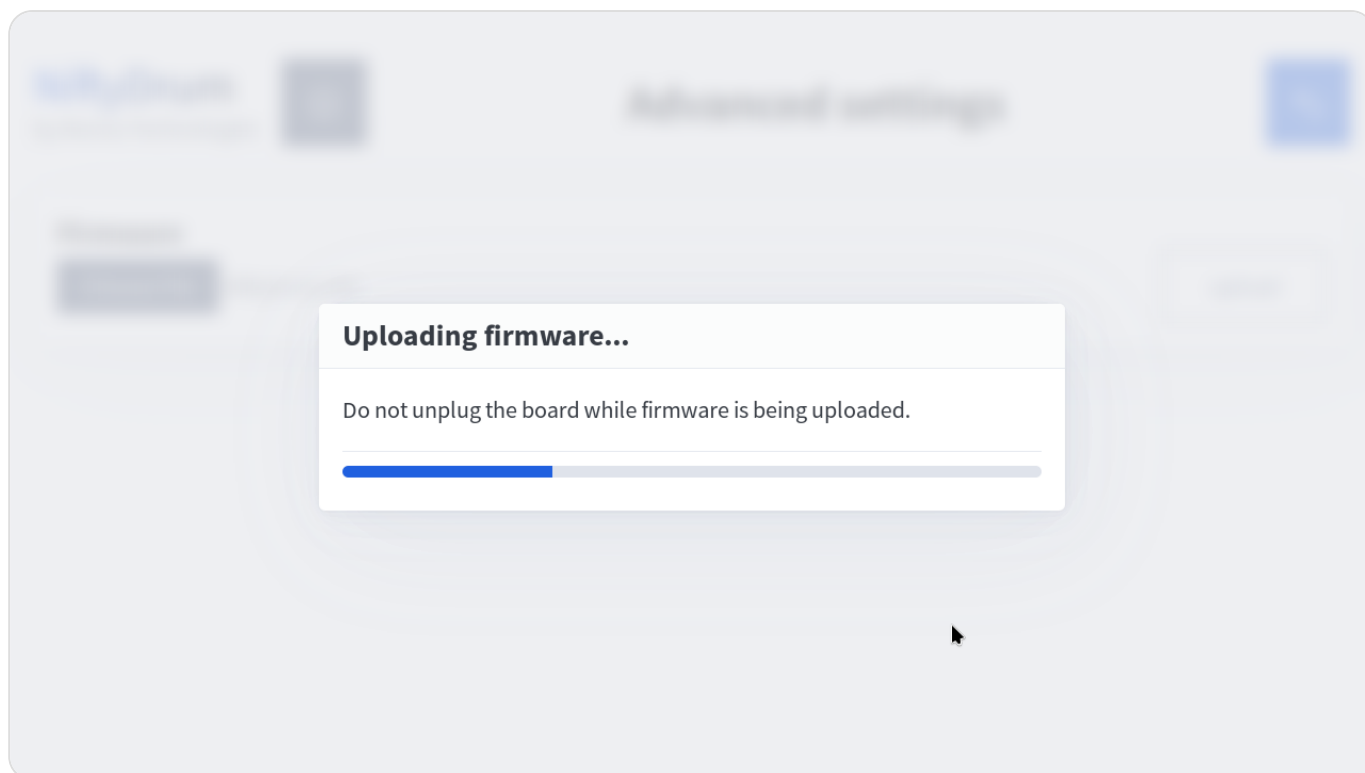
The app also supports **firmware upgrades** for your board.



The screenshot shows the 'Advanced settings' screen of the NiftyDrum app. At the top left is the 'NiftyDrum by Ronna Technologies' logo and an information icon. The title 'Advanced settings' is centered at the top, with a back arrow icon on the right. Below the title is a 'Firmware' section. It contains a file input field with a 'Choose File' button on the left and the filename 'NiftyDrum.bin' in the center. To the right of the file field is an 'upload' button with a mouse cursor hovering over it.

Here is how to upgrade the firmware:

- From the app's main screen, click the **"gear" icon** to access the **Advanced Settings** screen.
- Select a firmware file (if downloaded as a **.zip** , **unzip it first**).
- Ensure the file has a **.bin** extension and is an **officially supported firmware**.
- Click the **Upload** button and wait for the process to complete.



⚠ Warning

Do not unplug the board during the firmware upload.

🔥 Troubleshooting

If the firmware upload fails or the board is accidentally unplugged. Download [tkg-flash](#) . **Double-press the reset button** on the board. Upload the firmware via terminal:

Unix Windows

```
./tkg-flash NiftyDrum.bin
```

```
./tkg-flash.exe NiftyDrum.bin
```

Once the upload is complete, the board will **restart automatically**, and the app will **reconnect** to it.

4. Firmware Serial Protocol

4.1 Overview

The NiftyDrum board establishes a **virtual serial port over USB** when connected to a computer. It also features a **USART interface** on the Raspberry Pi-compatible GPIO port.

⚠ USART Voltage Limit

The USART pins are **3.3V only**. Exceeding this voltage may damage the board.

4.2 Command Syntax

All commands begin with a forward slash (/). Commands that require arguments follow the format:

```
/command arg1 arg2 arg3
```

4.2.1 Special Commands

Special commands do not require parameters. They either modify the board's behavior or provide specific information.

Command	Description
<code>/reset</code>	Reboots the board in bootloader mode.
<code>/factory_reset</code>	Resets all settings to factory defaults.
<code>/sn</code>	Returns the board's serial number.
<code>/version</code>	Returns the current firmware version.

4.2.2 Parameters Commands

The board uses **on-board EEPROM** to persist parameters. The serial protocol provides the following commands to save and load these parameters.

Command	Description
<code>/load params all</code>	Loads all parameters from EEPROM.
<code>/save params all</code>	Saves all current parameters to EEPROM.

4.2.3 Pad Parameters

Each pad supports **six configurable parameters**:

- Threshold
- Gain
- Scan time
- Mask time
- Decay
- MIDI note

Instrument Placeholder

The examples below use `snare`. Replace it with `hihat`, `kick`, `crash1`, `tom1`, `tom3`, `ride`, `tom2`, or `crash2` as needed.

Getters

Use the following commands to **retrieve** pad parameter values:

Command	Description	Example Reply
<code>/get snare threshold</code>	Gets the trigger threshold.	15
<code>/get snare gain</code>	Gets the sensor gain.	1.5
<code>/get snare scan</code>	Gets the scan time (μ s).	3000 (3ms)
<code>/get snare mask</code>	Gets the mask time (μ s).	20000 (20ms)
<code>/get snare decay</code>	Gets the decay time (μ s).	60000 (60ms)
<code>/get snare note</code>	Gets the MIDI note assigned to the pad.	36

Setters

Use the following commands to **set** pad parameter values:

Command	Description	Example Usage
<code>/set snare threshold</code>	Sets the trigger threshold.	<code>/set snare threshold 15</code>
<code>/set snare gain</code>	Sets the sensor gain.	<code>/set snare gain 1.5</code>
<code>/set snare scan</code>	Sets the scan time (μ s).	<code>/set snare scan 3000</code>
<code>/set snare mask</code>	Sets the mask time (μ s).	<code>/set snare mask 20000</code>
<code>/set snare decay</code>	Sets the decay time (μ s).	<code>/set snare decay 60000</code>
<code>/set snare note</code>	Sets the MIDI note for the pad.	<code>/set snare note 36</code>

⚠ Value Validation

The board **does not validate** input values. Incorrect values may cause unexpected behavior.

4.3 Hi-Hat Controller Parameters

The hi-hat controller operates with dual functionality:

- It sends CC (Control Change) messages when the pedal is pressed or released slowly.
- It triggers MIDI notes when the pedal movement is fast (e.g., a foot splash or chick).

These two sensors are denoted FSRC for FSR continuous, and FSRT for FSR trigger.

4.3.1 Continuous Parameters

Command	Description	Example Reply
<code>/get fsrc timeout</code>	Gets the minimum time interval between two CC messages (μ s).	9000
<code>/get fsrc threshold</code>	Gets FSR noise threshold.	30
<code>/get fsrc offset</code>	Gets the value from which the pedal is considered as fully closed.	105

Command	Description
<code>/set fsrc timeout</code>	Sets the minimum time interval between two CC messages (μ s).
<code>/set fsrc threshold</code>	Sets FSR noise threshold.
<code>/set fsrc offset</code>	Sets the value from which the pedal is considered as fully closed.

4.3.2 Trigger Parameters

Command	Description	Example Reply
<code>/get fsrt threshold</code>	Gets the minimum time interval between two CC messages (μ s).	9000
<code>/get fsrt scan</code>	Gets FSR noise threshold.	30
<code>/get fsrt velocity</code>	Gets the value from which the pedal is considered as fully closed.	105

Command	Description
<code>/set fsrt threshold</code>	Sets the minimum time interval between two CC messages (μ s).
<code>/set fsrt scan</code>	Sets FSR noise threshold.
<code>/set fsrt velocity</code>	Sets the value from which the pedal is considered as fully closed.

5. Sensor SDK

5.1 That's a Blue Pill

NiftyDrum is powered by the **STM32F103C8T6** microcontroller, the same chip found in the "**Blue Pill**" development board.

If you're unfamiliar with the Blue Pill, it's a widely recognized and versatile board based on the STM32F103C microcontroller family. With **72MHz** clock speed, **64KB of flash** memory, and **20KB of RAM**, it offers plenty of flexibility for customization and experimentation.

5.2 Arduino IDE

Since NiftyDrum uses the same microcontroller as the Blue Pill, you can program the board using the Arduino IDE.

5.2.1 Arduino STM32 SDK

Once you have a working ARM None-EABI toolchain, you will need to install the Arduino STM32 RonnaTech SDK.

Note

Although it's called Arduino STM32 RonnaTech SDK, this is actually a fork of [Roger Clark's Arduino STM32 repository](#) with some minor modifications.

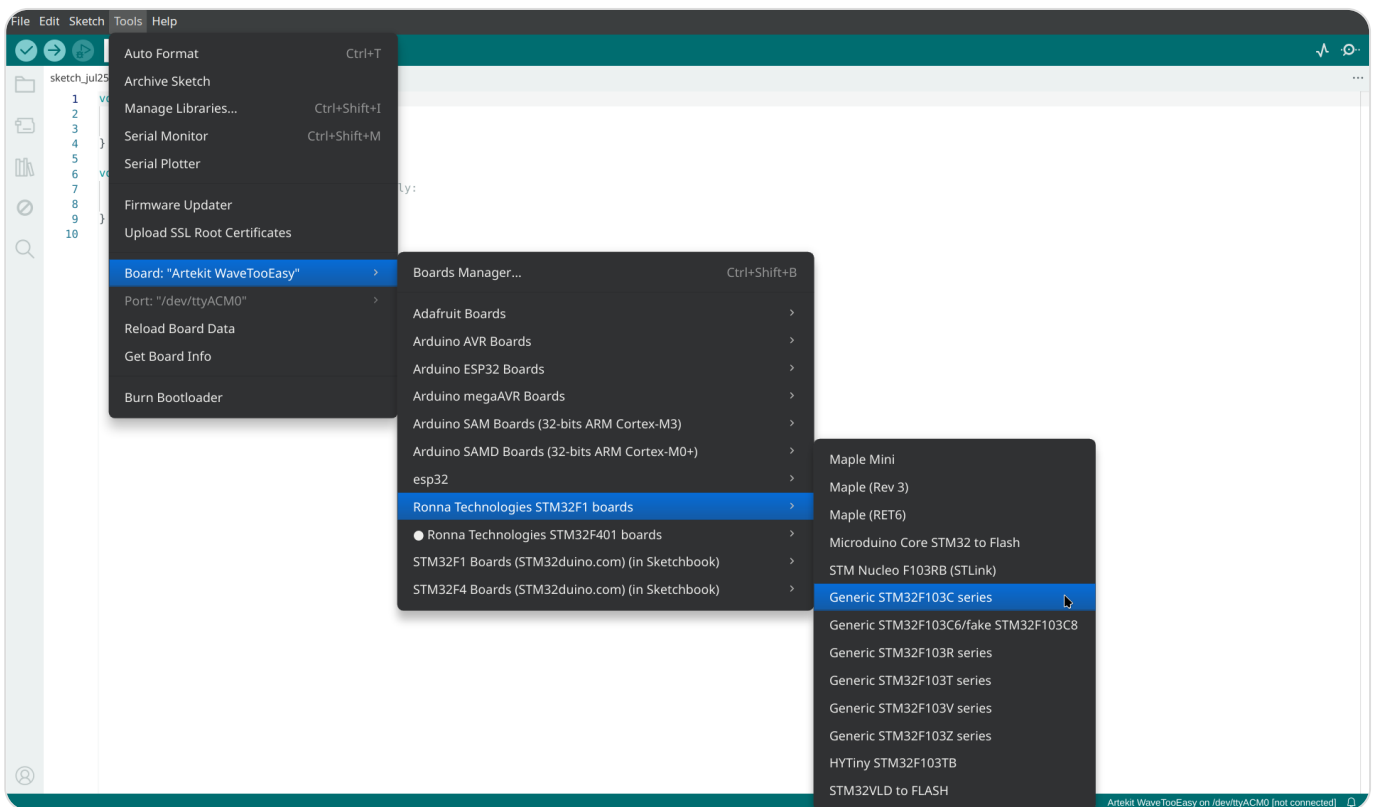
Here are the steps to follow to install the SDK and use it in the Arduino IDE:

1. Open **File > Preferences** in the Arduino IDE.
2. Add the following URL to the **Additional boards manager URLs** field:

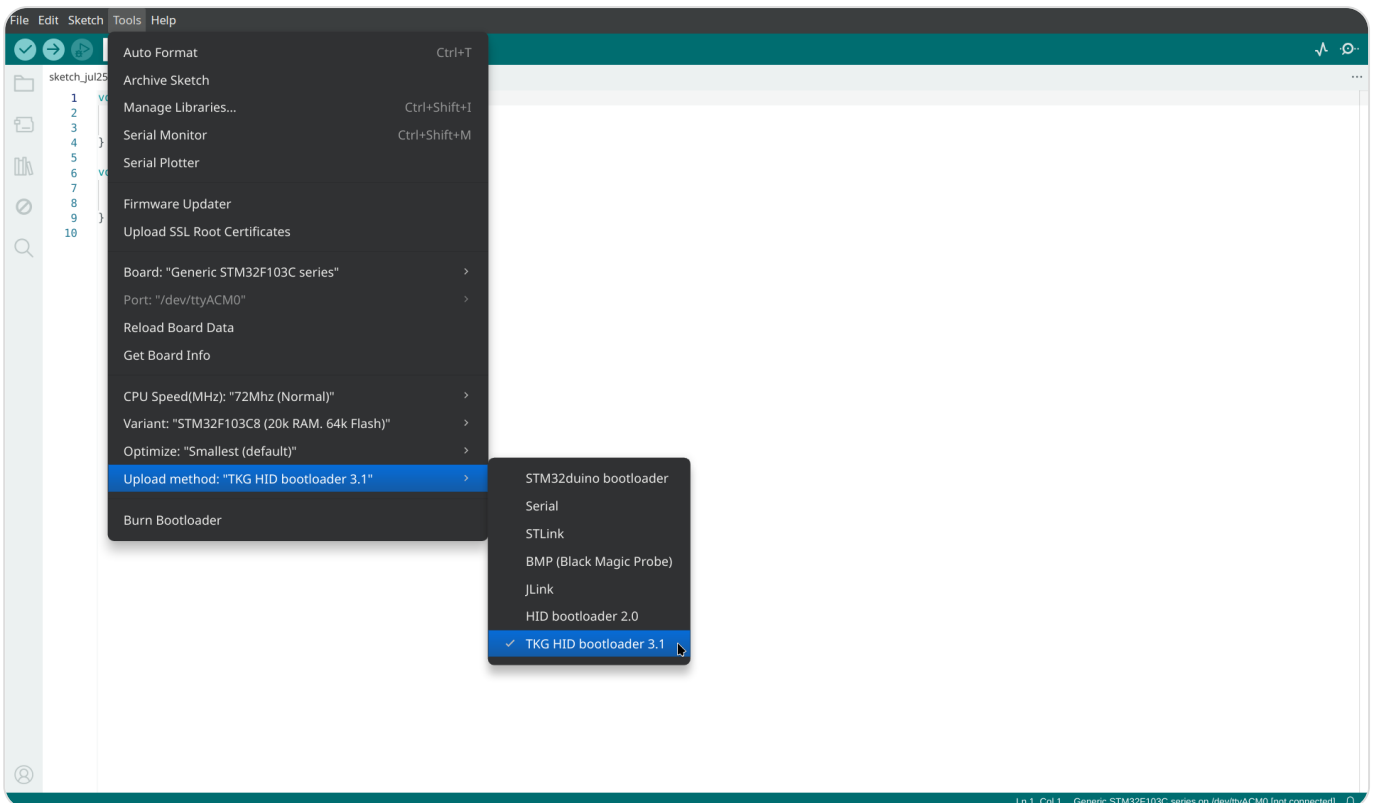
```
https://ronnatech.com/wp-content/uploads/package_ronnatech_index.json
```

3. Open the Boards Manager and search for RonnaTech.
4. Select **Ronna Technologies STM32F1** boards and install the latest version.

Now, go to the board manager and search for **stm32**. Once that is done, you can select the board from the menu:



And configure the IDE to use the correct board specs and upload method:



Be sure to select the correct CPU frequency, variant, and upload method, as shown on the image above. The most reliable way to build the firmware is to go to the **Sketch** menu and click on **Export Compiled Binary**, or use the shortcut `Alt + Ctrl + S`. Then, to find the firmware file, go to **Sketch** again, and **Show Sketch Folder**, or use the shortcut `Alt + Ctrl + K`.

To upload the firmware to the board, first enter bootloader mode on the board by double pressing the on-board button. The LED should blink fast. Download [tkg-flash](#). Load the firmware via terminal:

Unix Windows

```
./tkg-flash sketch.ino.bin
```

```
./tkg-flash.exe sketch.ino.bin
```

Once the upload is complete, the board will **restart automatically**, and the app will **reconnect** to it.

5.2.2 Arduino CLI

In this section, I will describe how to setup a configuration file to compile and upload a sketch using Arduino CLI. The binary is available from most platforms, but if you want to install it yourself you can [download it from GitHub](#).

Usually, I create a JSON file that holds the configuration I want. It looks like this:

arduino.json

```
{
  "configuration": "device_variant=STM32F103C8,upload_method=TKG-
  HIDUploadMethod,cpu_speed=speed_72mhz,opt=osstd",
  "board": "ronnatech:STM32F1:genericSTM32F103C",
  "sketch": "NiftyDrum.ino",
  "port": "/dev/ttyACM0",
  "output": "./output"
}
```

If you use [VSCode](#), you can put the file into a `.vscode` folder, and install the [Arduino Community Edition](#) extension, and it will work out of the box.

build.sh

```
#!/bin/bash

FQBN=$(jq -r '.board' arduino.json)
PORT=$(jq -r '.port' arduino.json)
CONFIG=$(jq -r '.configuration' arduino.json)
SKETCH=$(jq -r '.sketch' arduino.json)
OUTPUT=$(jq -r '.output' arduino.json)

rm -rf ./build/*

# Compile the sketch
./arduino-cli compile -v -v --fqbn "${FQBN}:${CONFIG}" --port $PORT $SKETCH --
build-path build --output-dir $OUTPUT
```

```
# Upload the sketch
./tkg-flash $OUTPUT/$SKETCH.bin
```

5.2.3 Blink Example

Writing code for the NiftyDrum board is as straightforward as programming any Arduino. Here's a simple blink example:

blink.ino

```
// The setup function runs once when you press reset or power the board
void setup() {
    // Initialize digital pin LED_BUILTIN as an output
    pinMode(LED_BUILTIN, OUTPUT);
}

// The loop function runs repeatedly forever
void loop() {
    digitalWrite(LED_BUILTIN, HIGH); // Turn the LED on
    delay(1000);                     // Wait for a second
    digitalWrite(LED_BUILTIN, LOW);  // Turn the LED off
    delay(1000);                     // Wait for a second
}
```

5.3 Sending MIDI Notes

midi.ino

```
#undef min
#undef max
#undef true
#undef false

#include <USBComposite.h>

USBCompositeSerial serial;
```

```

USBMIDI midi;

void setup()
{
    USBComposite.clear();

    USBComposite.setVendorId(0x0483);
    USBComposite.setProductId(0x5740);
    USBComposite.setProductString("NiftyDrum");
    USBComposite.setManufacturerString("Ronna Technologies");

    serial.registerComponent();
    serial.setTimeout(0);

    midi.registerComponent();

    USBComposite.begin();

    const auto br = 115'200;
    Serial3.begin(br, SERIAL_8N1);

    while (!USBComposite.isReady())
        ;

    pinMode(PC13, OUTPUT);
    digitalWrite(PC13, LOW);
}

void loop()
{
    for(;;)
    {
        midi.sendNoteOn(9, 38, 64);
        delayMicroseconds(100'000);
    }
}

```

6. Sound SDK

6.1 WaveTooEasy

The sound functionality of the board is based on **WaveTooEasy**, a polyphonic WAV player developed by [ArteKit Labs](#).

Key Differences in NiftyDrum+:

- Uses a **PCM5102A** audio DAC instead of the LM49450.
- The **ArteKit STM32F401 Core** has been slightly modified to include an analog input.
- **GCC 13.2** is used to provide **C++20** support.

6.2 RonnaTech STM32F401 Core SDK

The **RonnaTech STM32F401 Core SDK** is 99% identical to the **ArteKit STM32F401 Core SDK** and primarily targets the **STM32F401RCT6** microcontroller. It responds to **UART** for streaming sounds from a microSD card.

6.2.1 Installing the SDK

Here are the steps to follow to install the SDK and use it in the Arduino IDE:

1. Open **File > Preferences** in the Arduino IDE.
2. Add the following URL to the **Additional boards manager URLs** field:

```
https://ronnatech.com/wp-content/uploads/package_ronnatech_index.json
```

3. Open the Boards Manager and search for RonnaTech.
4. Select **Ronna Technologies STM32F401** boards and install the latest version.

6.3 Firmware Updates

6.4 Resources

6.5 Example: The Default Firmware
